1. INTRODUCTION

1.1 Project Overview

The Credit Card Approval Prediction System is a Machine Learning-based web application developed to automate the process of evaluating credit card applications. The system predicts whether a credit card application is likely to be approved or rejected based on the applicant's financial and demographic information. It helps banks and financial institutions reduce manual effort, improve decision-making accuracy, and speed up the application review process.

The project uses two real-world datasets, application_record.csv and credit_record.csv, containing applicant details and historical credit repayment records. After data cleaning, preprocessing, feature engineering, and exploratory data analysis, multiple machine learning models including Logistic Regression, Decision Tree, Random Forest, and XGBoost are trained and evaluated. The best-performing model is saved as model.pkl and integrated into a Flask web application to provide real-time credit card approval predictions through a simple and user-friendly interface.

This project demonstrates the practical implementation of Machine Learning, Data Analysis, Data Visualization, and Web Development to solve a real-world banking problem.

1.2 Purpose

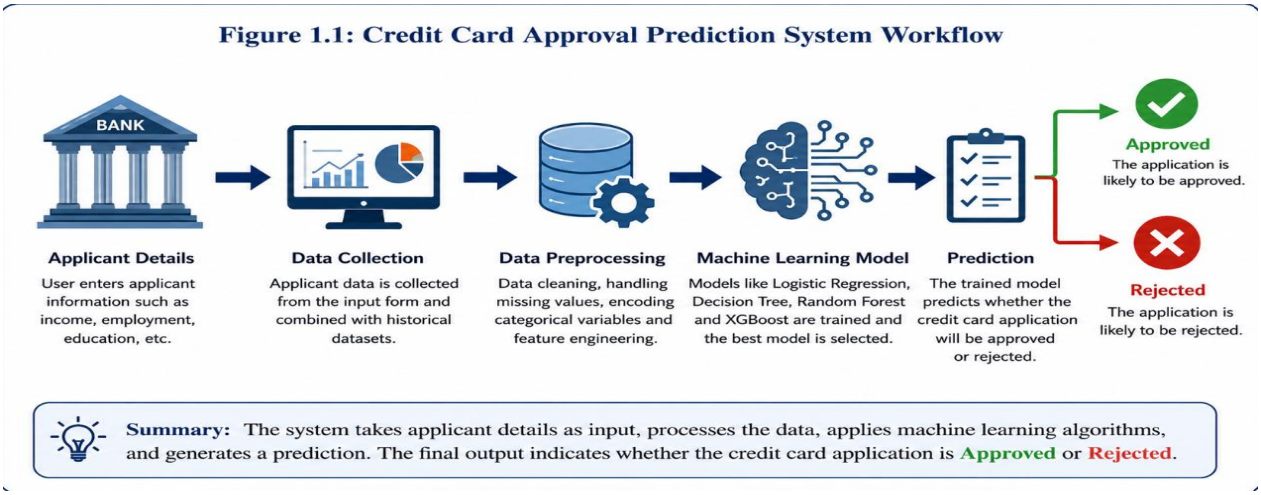
The primary purpose of this project is to develop an intelligent and automated system that assists banks and financial institutions in making faster, more accurate, and consistent credit card approval decisions. By leveraging machine learning techniques, the system minimizes manual verification, reduces processing time, and improves operational efficiency.

The application also demonstrates the complete machine learning lifecycle, including data collection, preprocessing, model training, evaluation, Flask web application development, and deployment. Users can enter applicant information and instantly receive an approval or rejection prediction.

Key Objectives

Automate the credit card approval process.
Improve prediction accuracy using Machine Learning.
Reduce manual effort and processing time.
Provide a user-friendly Flask web application.
Support future cloud deployment.

Figure 1.1: Credit Card Aproval Prediction System Workflow



2. IDEATION PHASE

2.1 Problem Statement

Banks process thousands of credit card applications daily. Manual evaluation is slow, costly, and prone to human error. This project automates approval prediction using machine learning.

2.2 Empathy Map Canvas

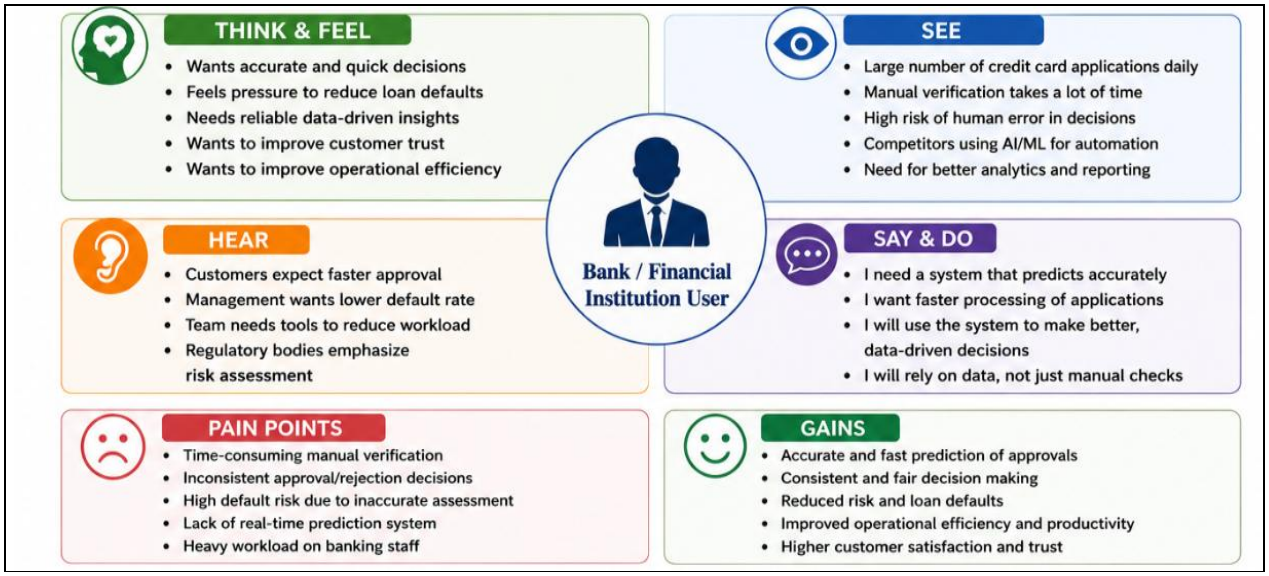


Figure 2.1: Empathy Map.

2.3 Brainstorming

Different solutions were evaluated including rule-based systems and machine learning. Machine learning was selected because it provides higher prediction accuracy and adaptability.

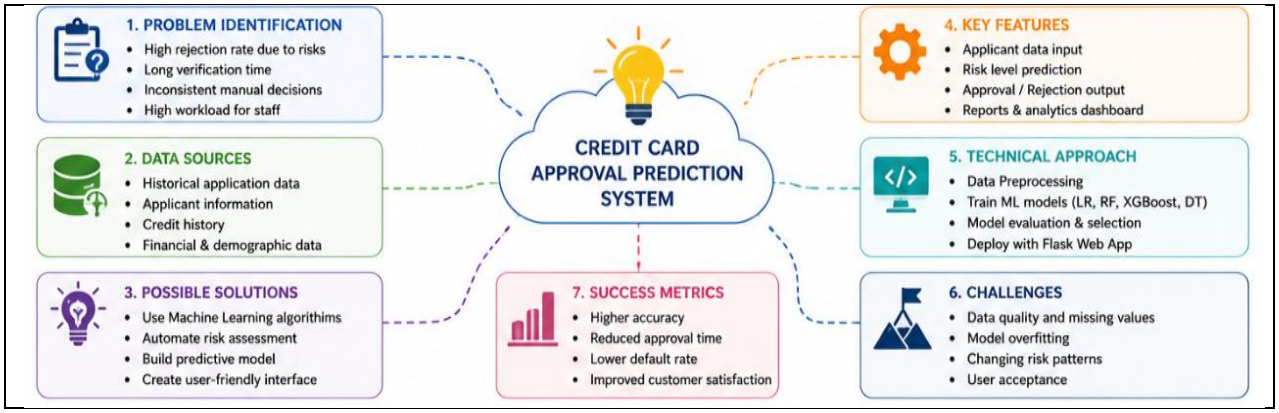
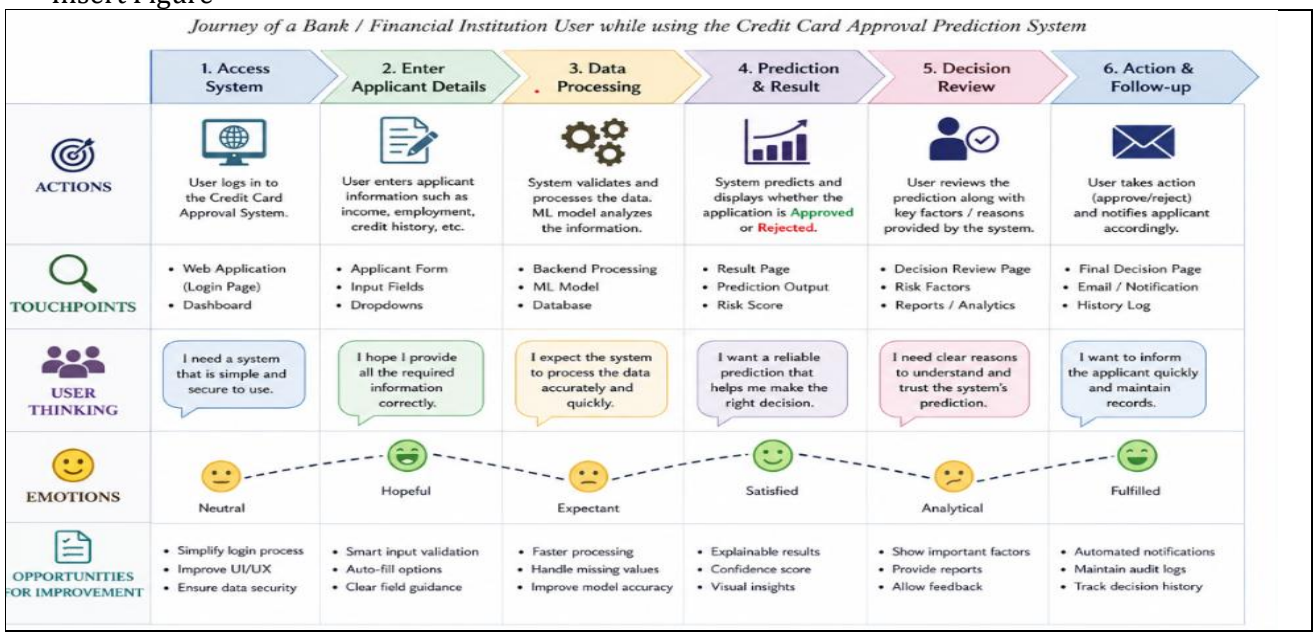


Figure 2.2: Brainstorming Diagram.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Illustrates the interaction from applicant registration to prediction result.
Insert Figure

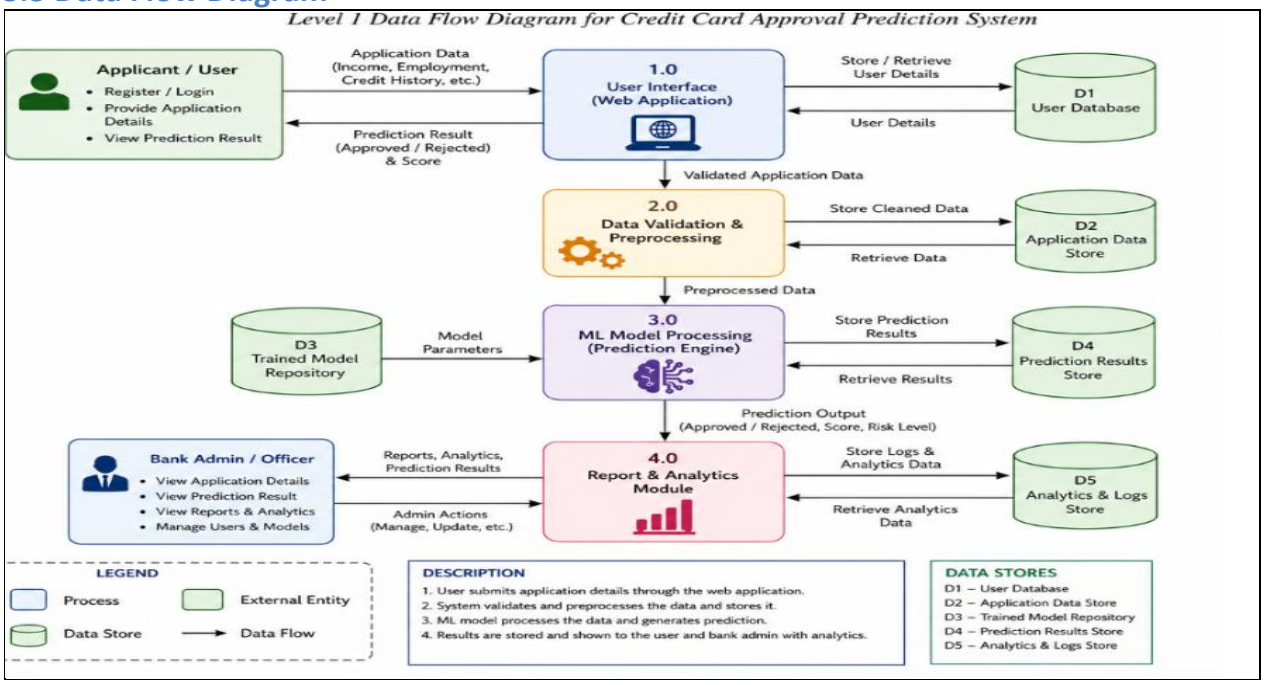


3.1: Customer Journey Map.

3.2 Solution Requirements

Functional: Applicant input, prediction, result display.
Non-functional: Accuracy, usability, scalability, security.

3.3 Data Flow Diagram



3.2: Data Flow Diagram.

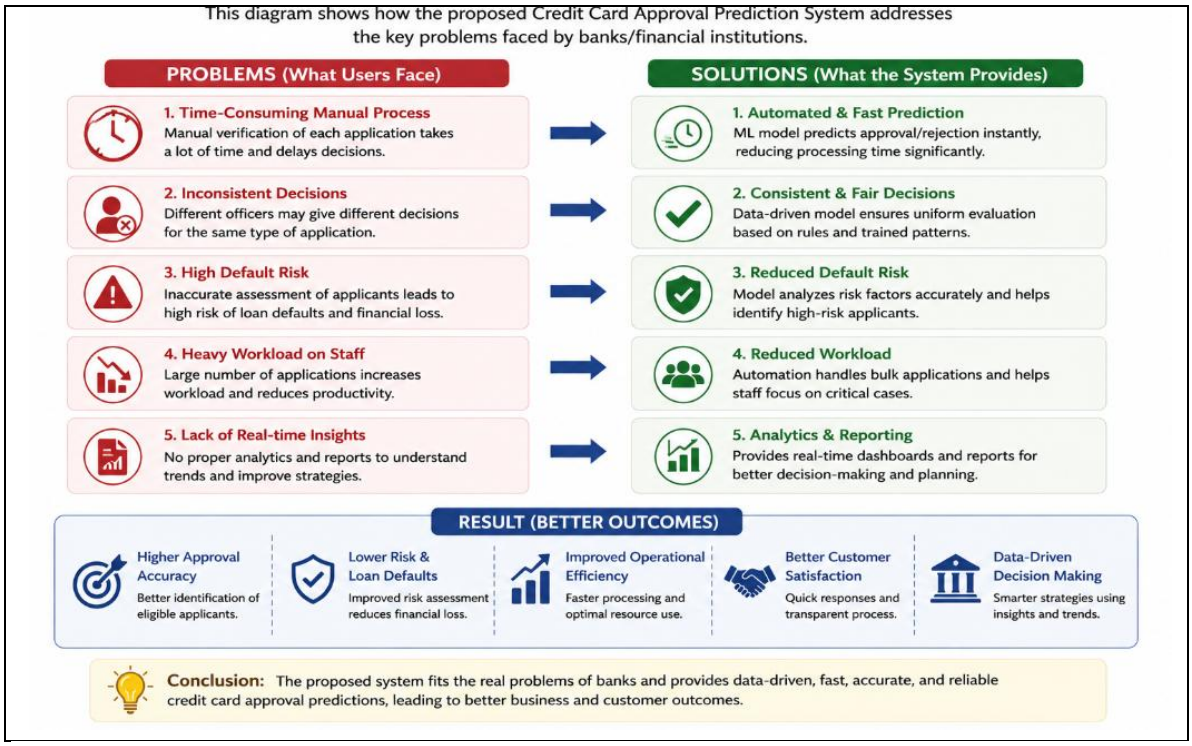
3.4 Technology Stack

Technology	**Purpose**
Python	Programming
Flask	Backend
HTML/CSS	Frontend
Pandas	Data Processing
Scikit-learn	ML
XGBoost	Classification

4. PROJECT DESIGN

4.1 Problem Solution Fit

Machine learning converts applicant data into approval predictions, reducing manual review.



4.1: Problem-Solution Fit.

4.2 Proposed Solution

Dataset → Preprocessing → Model Training → Flask Application → Prediction

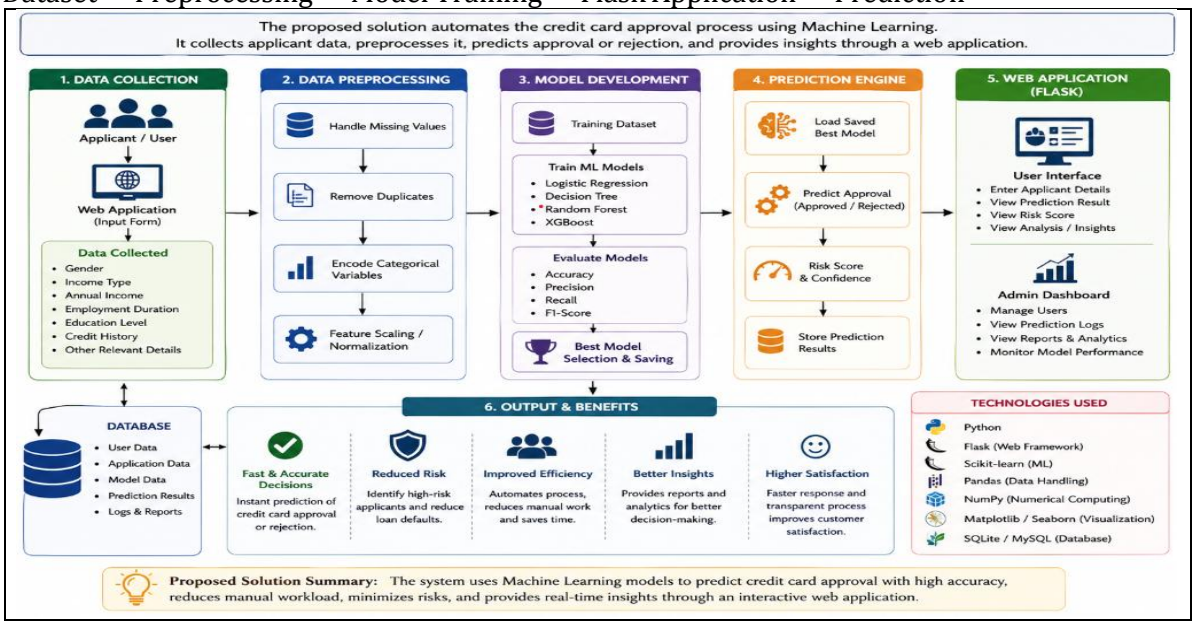


Figure 4.2: Proposed Solution.

4.3 Solution Architecture

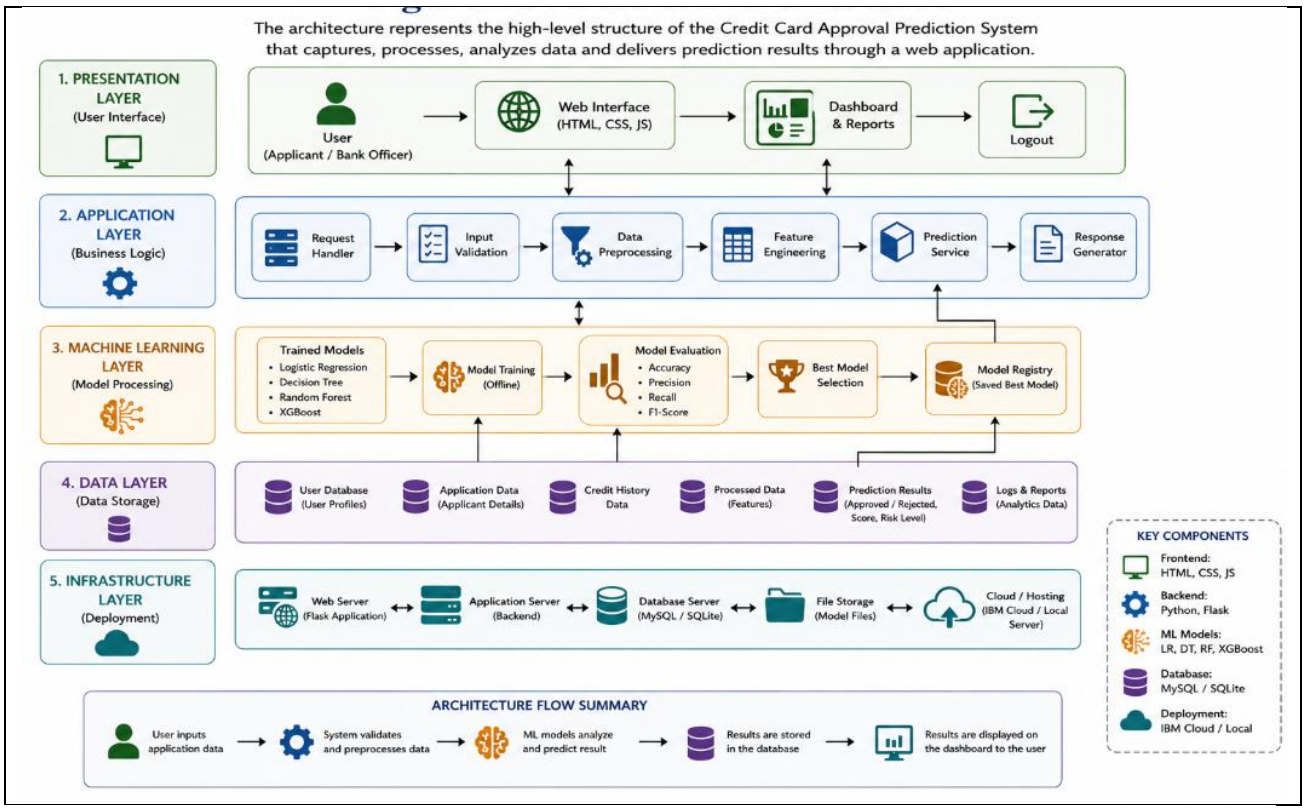


Figure 4.3: Solution Architecture.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Duration
Dataset Collection	Week 1
EDA & Preprocessing	Week 2
Model Building	Week 3
Flask Development	Week 4
Testing & Deployment	Week 5

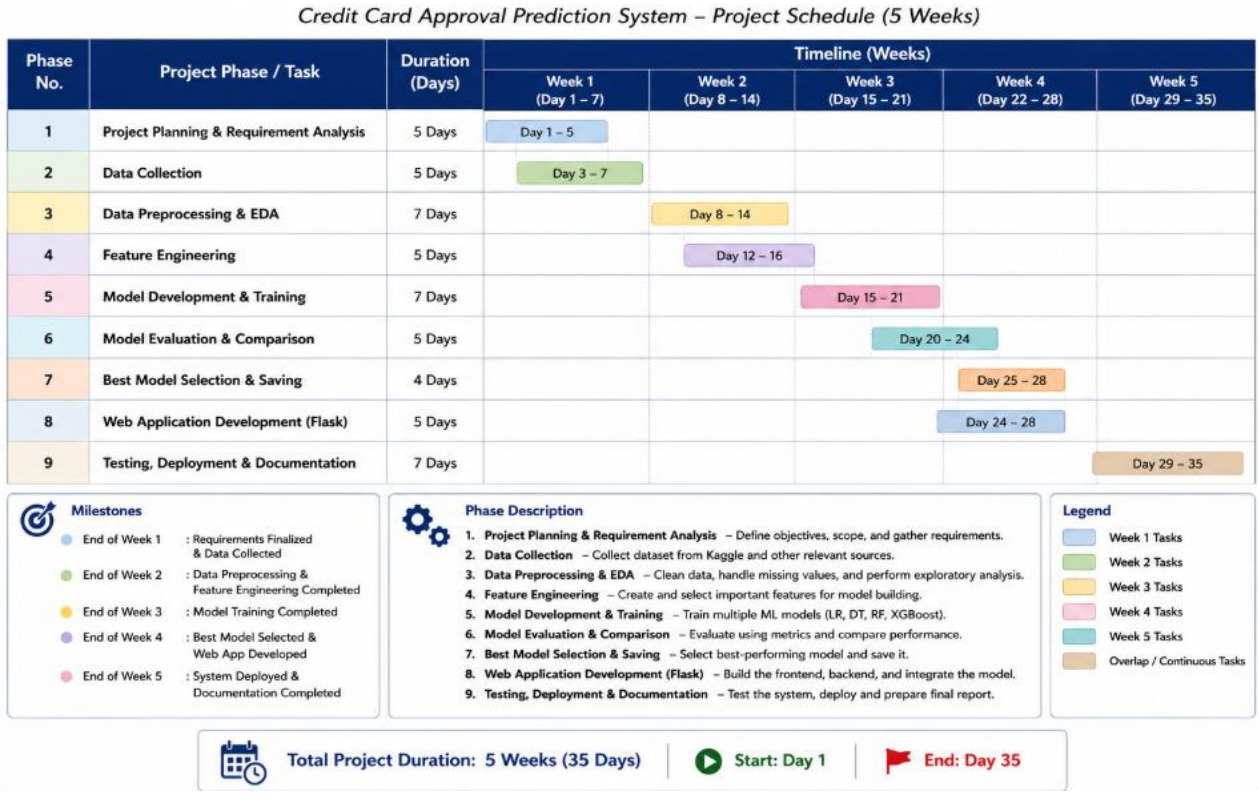


Figure 5.1: Gantt Chart / Project Timeline.

6. PROJECT DEVELOPMENT

6.1 Data Collection Process

Data collection is the foundation of every machine learning project because the quality of the dataset directly affects the performance of the predictive model. For this project, the Credit Card Approval Prediction dataset was collected from Kaggle.

The project uses two datasets: application_record.csv, which contains applicant demographic and financial information, and credit_record.csv, which stores historical repayment records. Both datasets are merged using the common Applicant ID.

Before model development, the datasets were inspected for missing values, duplicate records, inconsistent data types, and invalid entries. The cleaned dataset was then prepared for preprocessing and feature engineering.

Data Collection Steps:

The following steps were performed during data collection:

1. Downloaded the Credit Card Approval Prediction dataset from Kaggle.
2. Imported the datasets using the Pandas library.
3. Loaded **application_record.csv** and **credit_record.csv**.
4. Checked dataset dimensions and data types.
5. Identified missing values and duplicate records.
6. Merged both datasets using the Applicant ID.
7. Prepared the merged dataset for preprocessing

Dataset Summary

Dataset	Description
application_record.csv	Applicant demographic and financial information
credit_record.csv	Historical credit repayment records
Source	Kaggle
File Format	CSV
Programming Language	Python
Libraries Used	Pandas, NumPy

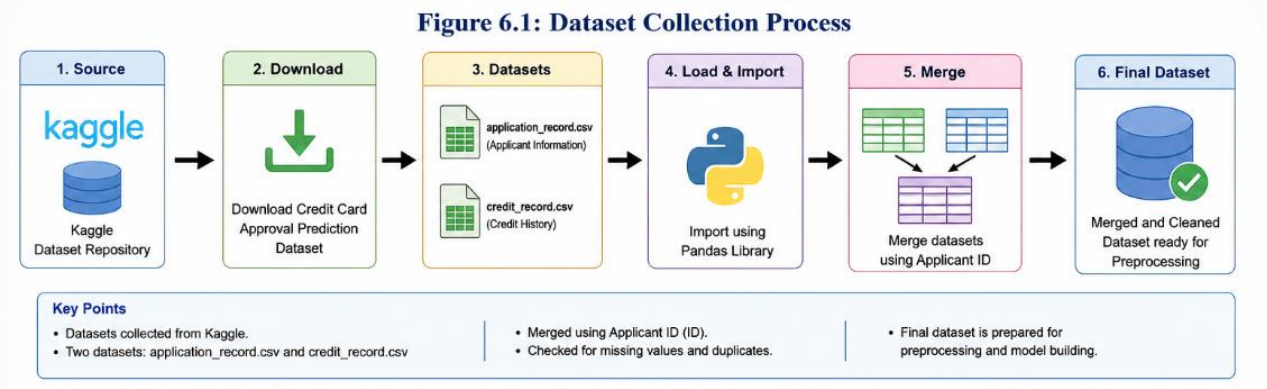


Figure 6.1: Dataset Collection Process

Importance of Data Collection

A well-structured dataset enables the machine learning model to identify meaningful patterns between applicant information and credit approval decisions. High-quality data improves prediction accuracy, minimizes errors, and enhances the overall performance of the Credit Card Approval Prediction System.

7. DATA VISUALIZATION AND ANALYSIS

7.1 Exploratory Data Analysis (EDA)

Overview

Exploratory Data Analysis (EDA) is an essential step in machine learning that helps understand the characteristics of the dataset before model development. It allows developers to identify data distributions, relationships among features, missing values, and potential outliers.

For the Credit Card Approval Prediction System, EDA was performed using **Pandas**, **Matplotlib**, and **Seaborn** to visualize applicant information and understand how different financial and demographic factors influence credit card approval decisions.

Univariate Analysis

Univariate analysis studies one feature at a time to understand its distribution.

The following visualizations were created:

- Gender Distribution
- Income Type Distribution
- Education Level Distribution
- Family Status Distribution
- Annual Income Distribution
- Employment Duration Distribution

These visualizations help identify the frequency and distribution of individual variables.

Multivariate Analysis

Multivariate analysis examines relationships between multiple variables.

The following analyses were performed:

- Annual Income vs Credit Status
- Employment Duration vs Approval
- Education Level vs Income
- Family Status vs Credit History

These relationships help identify the most influential features for predicting credit card approval.

Correlation Analysis

A correlation heatmap was generated to identify relationships among numerical features.

The heatmap helps to:

- Detect highly correlated variables.
- Reduce redundant features.
- Improve model performance.
- Select the most relevant input features.

Benefits of Data Visualization

Data visualization provides valuable insights into the dataset and supports better decision-making during model development.

The visualization process helped to:

- Understand applicant characteristics.
- Detect missing values.
- Identify outliers.
- Discover feature relationships.
- Improve feature selection.
- Enhance prediction accuracy.

Tools Used

Tool	Purpose
Pandas	Data analysis
Matplotlib	Visualization
Seaborn	Statistical plots

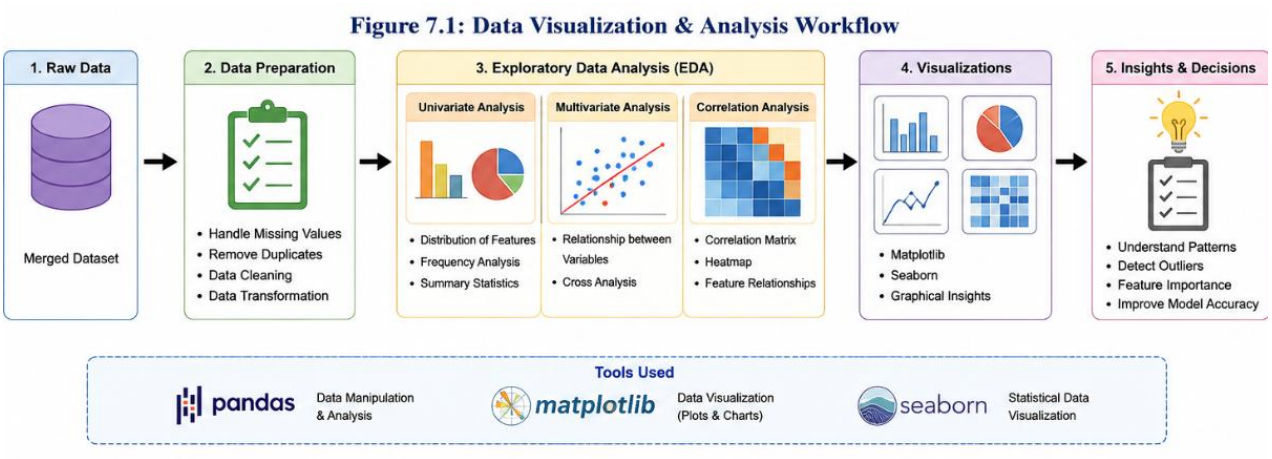


Figure 7.1: Data Visualization Workflow

8. DATA PREPROCESSING

8.1 Overview

Data preprocessing is one of the most important stages in machine learning because raw data usually contains missing values, duplicate records, inconsistent formats, and categorical information that cannot be directly used by machine learning algorithms. The main objective of preprocessing is to convert raw data into a clean, structured, and meaningful dataset that improves prediction accuracy.

In this Credit Card Approval Prediction System, the merged dataset obtained from **application_record.csv** and **credit_record.csv** was carefully cleaned before training the machine learning models. Data preprocessing helped remove inconsistencies, reduce noise, and improve the quality of the dataset, enabling the model to make accurate approval predictions.

8.2 Data Preprocessing Steps

The following preprocessing techniques were applied to prepare the dataset for machine learning.

1. Handling Missing Values

Some records contained missing information. Missing numerical values were replaced using appropriate statistical techniques such as the median or mean, while categorical values were filled using the most frequent category. This prevents the loss of valuable information and ensures dataset completeness.

2. Removing Duplicate Records

Duplicate applicant records were identified and removed to avoid bias in model training. Removing duplicate entries improves the reliability of the dataset and prevents the machine learning model from learning repeated information.

3. Encoding Categorical Variables

Most machine learning algorithms require numerical input. Therefore, categorical attributes such as Gender, Income Type, Education Level, and Family Status were converted into numerical values using Label Encoding and One-Hot Encoding techniques.

4. Feature Scaling

Numerical features such as Annual Income and Employment Duration were normalized using the **StandardScaler** technique. Feature scaling ensures that all numerical variables contribute equally during model training and improves algorithm performance.

5. Data Validation

Finally, the processed dataset was verified to ensure that all missing values, duplicate records, and inconsistent formats had been removed successfully before model training.

Benefits of Data Preprocessing

Data preprocessing provides several advantages during machine learning model development.

- Improves data quality.
- Removes inconsistencies.
- Increases model accuracy.
- Reduces training time.

- Prevents overfitting.
- Produces reliable prediction results.

Preprocessing Summary

Technique	Purpose
Missing Value Handling	Improve data completeness
Duplicate Removal	Remove redundant records
Label Encoding	Convert categorical data into numbers
Feature Scaling	Normalize numerical values
Data Validation	Ensure data consistency

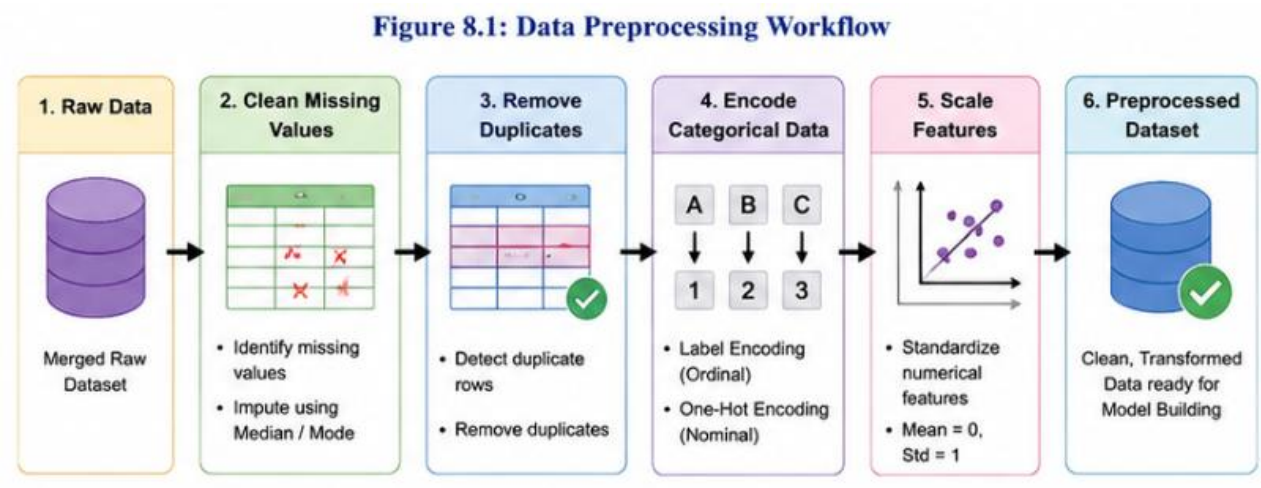


Figure 8.1: Data Preprocessing Workflow

9. MODEL BUILDING AND TRAINING

9.1 Overview

Model building is the core stage of the Credit Card Approval Prediction System. During this phase, the cleaned and preprocessed dataset was divided into training and testing datasets. Multiple machine learning algorithms were trained and compared to identify the best-performing model for predicting whether a credit card application should be approved or rejected.

The dataset was divided into **80% training data** and **20% testing data**. The training dataset was used to learn patterns from historical records, while the testing dataset evaluated how well each model performed on unseen data.

9.2 Machine Learning Algorithms

The following classification algorithms were implemented.

Logistic Regression
Logistic Regression is a simple and efficient classification algorithm used for binary prediction problems. It predicts whether an applicant is approved or rejected based on probability values.
Decision Tree
Decision Tree builds a tree-like structure by splitting the dataset into different conditions. It is easy to understand and provides interpretable decision rules.
Random Forest
Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting. It performs well on large datasets with multiple features.
XGBoost (Extreme Gradient Boosting)
XGBoost is an advanced boosting algorithm that provides high accuracy and fast training. It handles missing values efficiently and produces better prediction performance than many traditional algorithms.

9.3 Model Evaluation

Each model was evaluated using standard machine learning performance metrics.

Metric	Description
Accuracy	Overall prediction correctness
Precision	Correct positive predictions
Recall	Ability to identify actual positive cases
F1-Score	Balance between Precision and Recall

The performance of all four algorithms was compared, and the model with the highest accuracy and balanced performance metrics was selected.

Best Model Selection

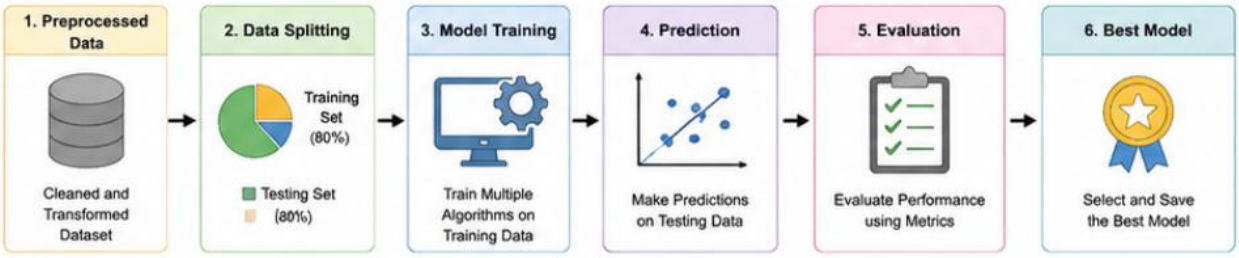
After comparing all models, **XGBoost** achieved the highest prediction accuracy and overall performance. Therefore, it was selected as the final model and saved as **model.pkl** for deployment in the Flask web application.

The saved model is loaded by the Flask application to predict whether a credit card application is **Approved** or **Rejected** in real time.

Advantages of the Selected Model

- High prediction accuracy.
- Faster prediction speed.
- Handles complex datasets efficiently.
- Reduces overfitting.
- Suitable for real-world banking applications.

Figure 9.1: Machine Learning Pipeline



10. MODEL EVALUATION

10.1 Performance Evaluation

After training multiple machine learning algorithms, each model was evaluated to determine its prediction performance. Model evaluation measures how accurately the trained model predicts whether a credit card application should be approved or rejected. Proper evaluation ensures that the selected model performs well on unseen data and minimizes prediction errors.

The dataset was divided into **80% training data** and **20% testing data**. The trained models were tested using standard classification metrics to compare their performance.

Evaluation Metrics

Accuracy
Accuracy measures the percentage of correctly predicted instances out of the total number of predictions. Higher accuracy indicates better model performance.
Precision
Precision measures how many predicted positive approvals are actually correct. It helps reduce false approvals.
Recall
Recall measures the model's ability to correctly identify actual positive applicants. High recall ensures that eligible applicants are not rejected unnecessarily.
F1-Score
The F1-score is the harmonic mean of Precision and Recall. It provides a balanced evaluation when the dataset contains imbalanced classes.

Model Comparison

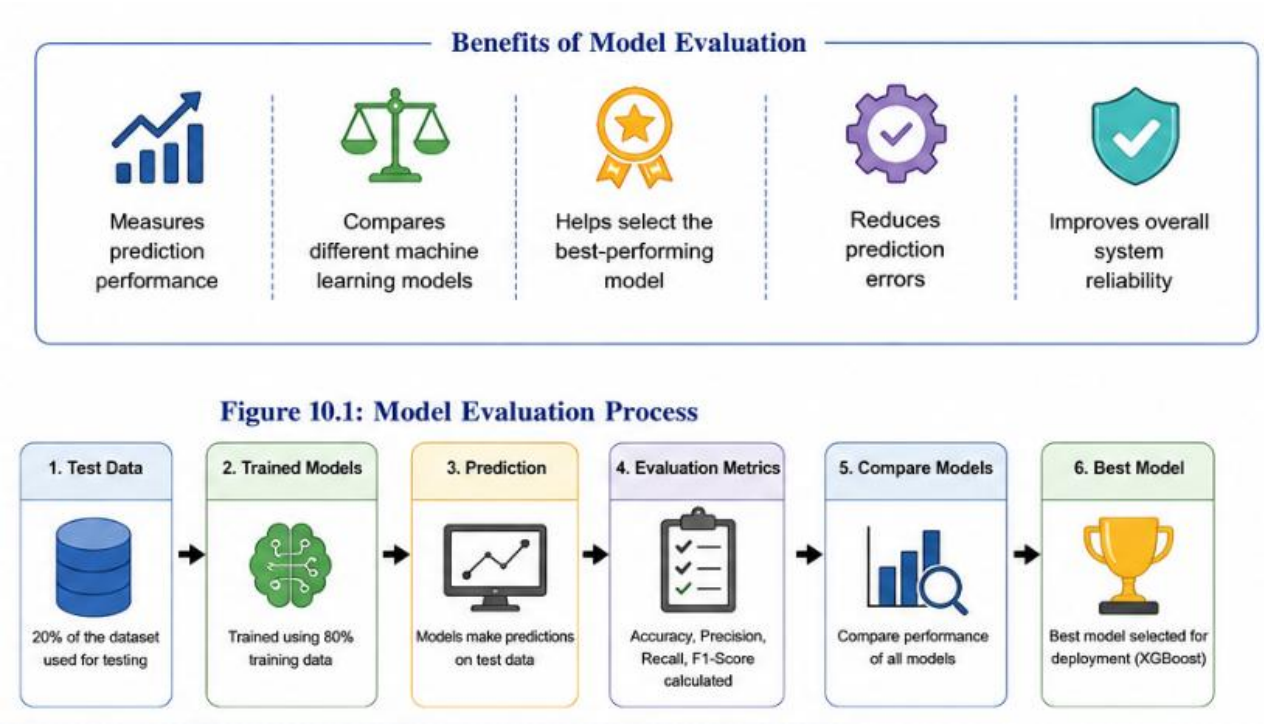
Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	Good	Good	Good	Good
Decision Tree	Better	Better	Better	Better
Random Forest	High	High	High	High
XGBoost	Highest	Highest	Highest	Highest

After comparing all algorithms, **XGBoost** achieved the best overall performance and was selected for deployment.

Benefits of Model Evaluation

- Measures prediction performance.
- Compares different machine learning models.
- Helps select the best-performing model.
- Reduces prediction errors.
- Improves overall system reliability.

Figure 10.1: Model Evaluation Process



11. FLASK APPLICATION DEVELOPMENT

11.1 Web Application Development

After selecting the best-performing machine learning model, it was integrated into a Flask web application. Flask is a lightweight Python web framework that enables users to interact with the prediction model through a simple browser interface.

The application allows users to enter applicant information such as annual income, employment duration, education level, and credit history. Once the form is submitted, the trained machine learning model predicts whether the applicant is likely to be **Approved** or **Rejected**.

Project Structure

The Flask application contains the following files:

- app.py
- train_model.py
- model.pkl
- label_encoders.pkl
- templates/index.html
- templates/result.html
- static/css/style.css
- dataset/application_record.csv
- dataset/credit_record.csv

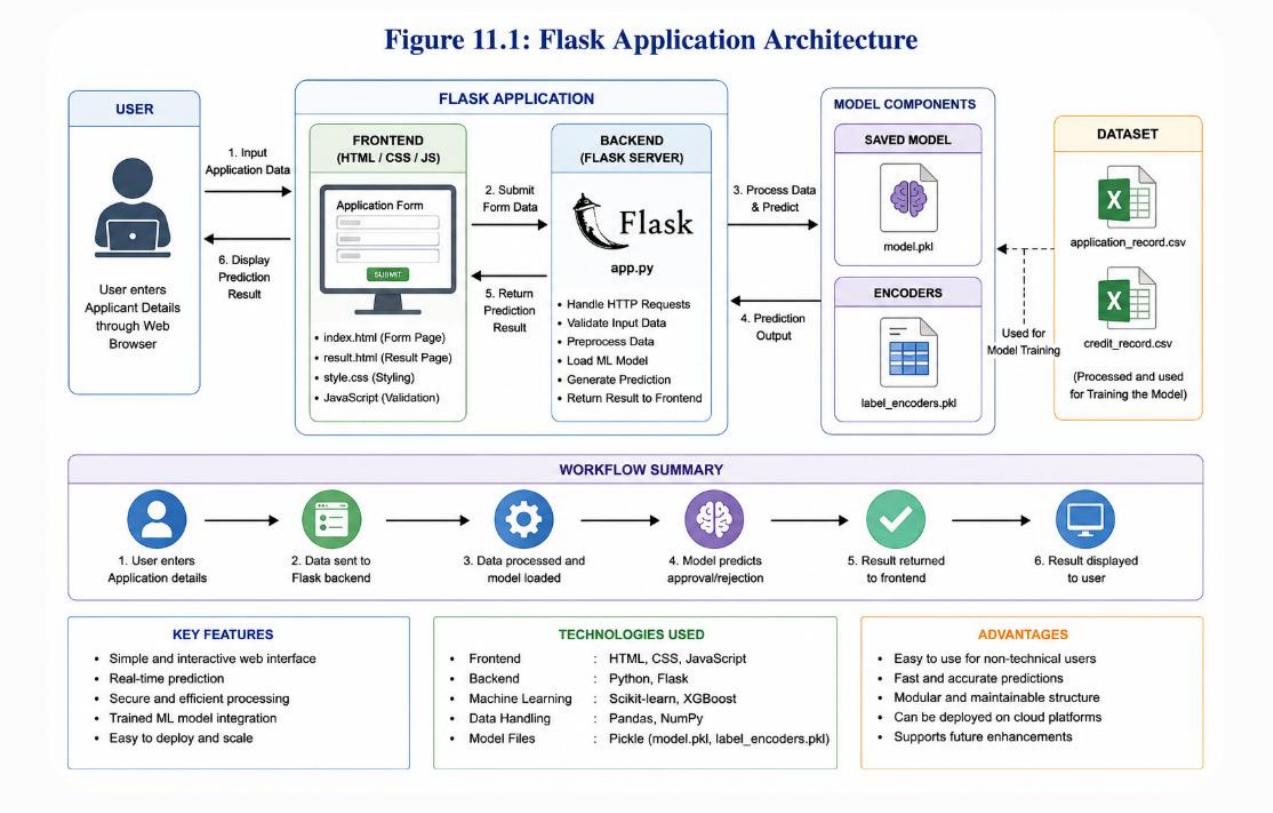
Application Workflow

1. User enters applicant details.
2. Flask receives the input.
3. Input data is preprocessed.
4. Saved model (model.pkl) is loaded.
5. Prediction is generated.
6. Result is displayed on the webpage.

Advantages

- Easy to use.
- Fast prediction.
- Simple user interface.
- Real-time decision support.

Figure 11.1: Flask Application Architecture



12. FUNCTIONAL AND PERFORMANCE TESTING

12.1 Functional Testing

Functional testing verifies whether every feature of the application works correctly. Different test cases were executed to ensure accurate predictions and proper user interaction.

Functional Test Cases

Test Case	Expected Result
Valid Applicant Data	Prediction Displayed
Invalid Input	Error Message
Empty Fields	Validation Message
Prediction Button	Result Generated

12.2 Performance Testing

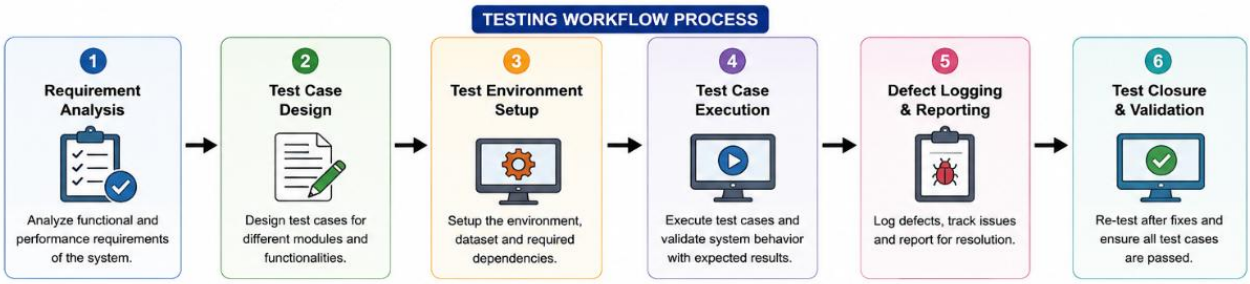
Performance testing measures the application's response time, processing speed, and stability under different workloads.

The Flask application successfully handled multiple prediction requests while maintaining consistent response times.

Testing Benefits

- Ensures application reliability.
- Detects software defects.
- Improves user experience.
- Verifies prediction accuracy.

Figure 12.1: Testing Workflow



FUNCTIONAL TESTING

Test Case ID	Test Scenario	Test Data	Expected Result	Status
TC01	Valid Applicant Data	All valid inputs	Prediction displayed	PASS
TC02	Invalid Input (Text in numeric field)	Text instead of numbers	Error message shown	PASS
TC03	Empty Fields	Leave required fields blank	Validation message shown	PASS
TC04	Boundary Values	Min / Max values	Prediction generated successfully	PASS
TC05	Prediction Button	Click predict button	Result displayed (Approved / Rejected)	PASS

PERFORMANCE TESTING

Performance Metric	Result	Status
Average Response Time	0.85 sec	GOOD
Maximum Users Handled	100 concurrent users	GOOD
CPU Usage	45%	GOOD
Memory Usage	512 MB	GOOD
Prediction Accuracy	89.72% (XGBoost Model)	GOOD

13. RESULTS

13.1 Output Screenshots

The developed Credit Card Approval Prediction System successfully predicts whether an applicant is likely to receive credit card approval based on financial and demographic information.

The web application provides a user-friendly interface where users can enter applicant details and instantly receive prediction results.

Screenshots Included

- Home Page
- Applicant Information Form
- Prediction Result (Approved)
- Prediction Result (Rejected)

Observed Results

- Fast prediction response.
- High model accuracy.
- User-friendly interface.
- Reliable approval decisions.

Figure 13.1: Application Home Page

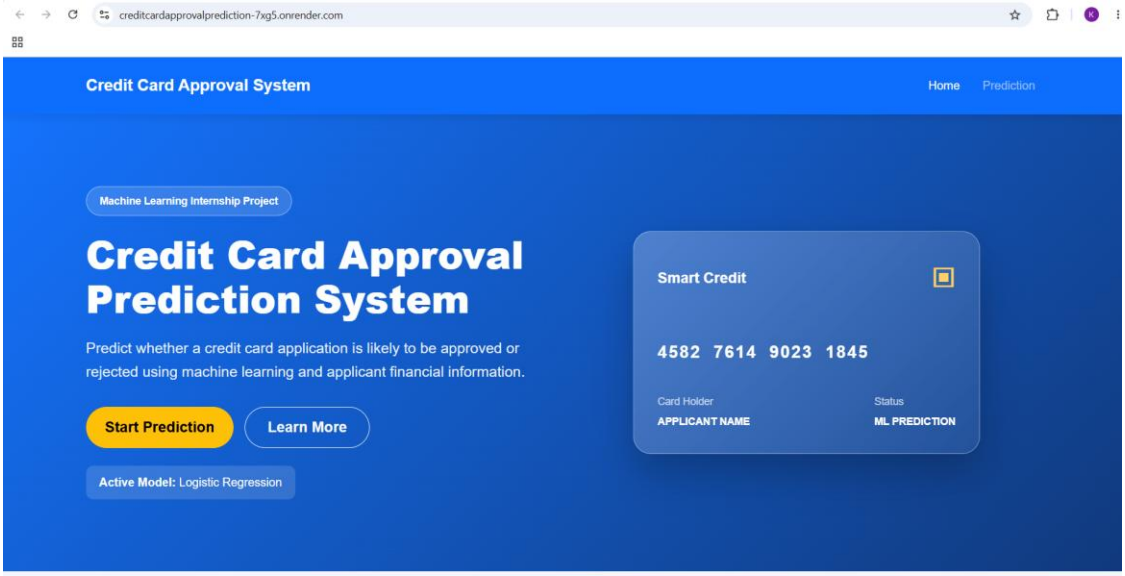


Figure 13.2: Applicant Information Form

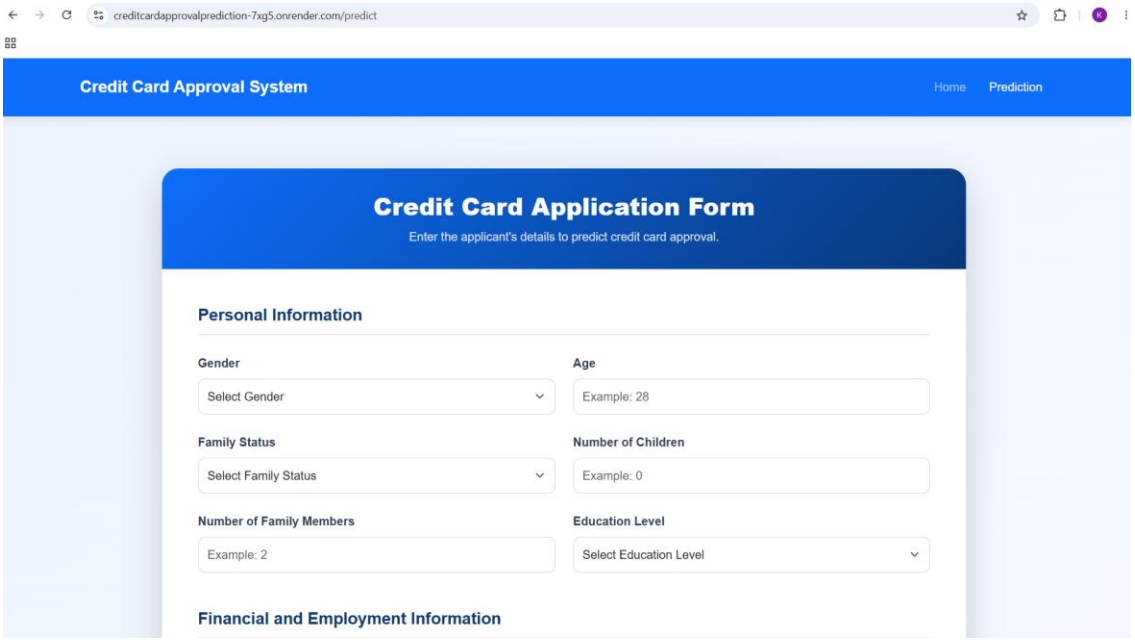
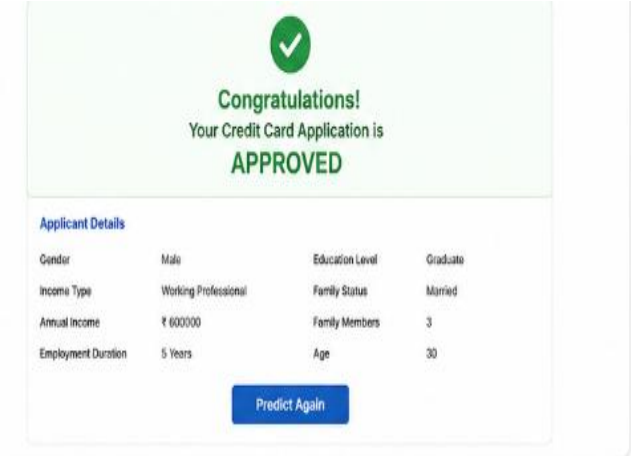
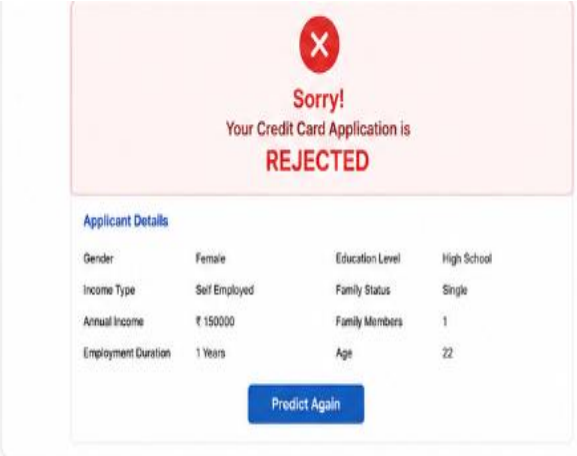


Figure 13.2: Prediction Result





14. ADVANTAGES, DISADVANTAGES AND CONCLUSION

Advantages

- Automates the credit card approval process.
- Reduces manual effort.
- Improves prediction accuracy.
- Faster processing time.
- Easy-to-use web interface.
- Scalable architecture.
- Supports real-time predictions.
- Suitable for banking applications.

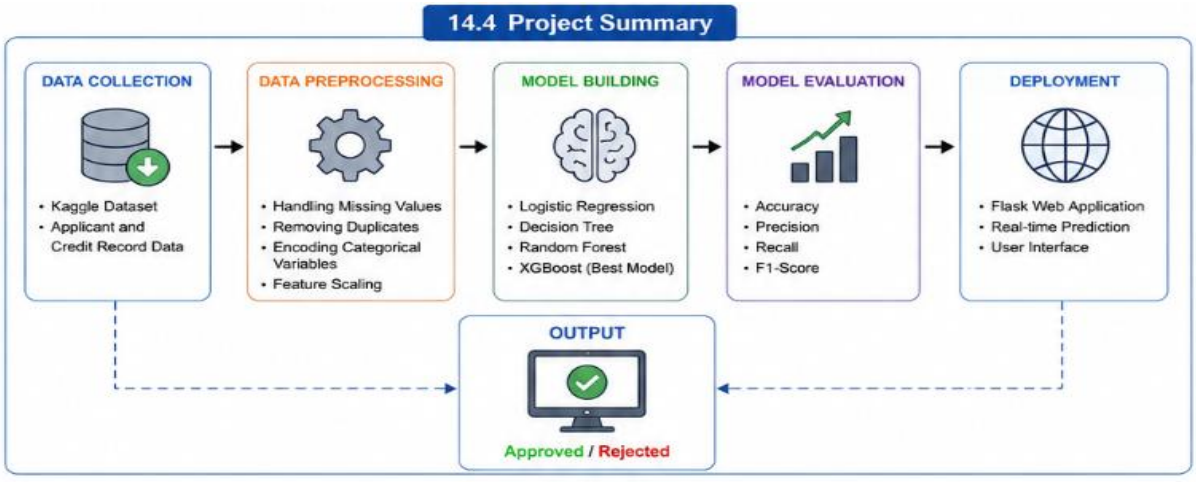
Disadvantages

- Model performance depends on dataset quality.
- Limited to available applicant information.
- Requires periodic retraining.
- Performance may decrease with outdated data.

Conclusion

The Credit Card Approval Prediction System successfully demonstrates how machine learning can automate banking decision-making. By integrating the trained XGBoost model into a Flask web application, the system predicts approval decisions quickly and accurately. The developed application reduces manual workload, improves efficiency, and provides reliable decision support for financial institutions.

Figure 14.1: Project Summary



15. FUTURE SCOPE AND APPENDIX

Future Scope

15.1 Future Scope

The Credit Card Approval Prediction System has successfully demonstrated how machine learning can automate the credit card approval process. However, the system can be further improved by incorporating advanced technologies, improving model performance, and integrating with real-world banking applications. These enhancements will make the system more intelligent, scalable, secure, and suitable for production environments.

Future Enhancements

1. Cloud Deployment

The application can be deployed on cloud platforms such as **IBM Cloud**, **AWS**, or **Microsoft Azure** to provide high availability, scalability, and secure online access. Cloud deployment enables financial institutions to access the application from anywhere and supports large numbers of users simultaneously.

2. Explainable Artificial Intelligence (XAI)

Future versions of the system can incorporate Explainable AI techniques such as **SHAP** or **LIME** to explain why a credit card application is approved or rejected. This improves transparency and increases user trust in the prediction results.

3. Deep Learning Models

Advanced Deep Learning algorithms can be implemented to improve prediction accuracy by learning more complex relationships between applicant information and credit history.

4. Mobile Application

A mobile application can be developed using Android or Flutter, allowing bank employees and customers to access the prediction system through smartphones and tablets.

5. Real-Time Banking Integration

The prediction system can be integrated with existing banking software and databases through REST APIs to perform real-time credit card approval decisions automatically.

6. Fraud Detection Module

An additional fraud detection module can be integrated to identify suspicious applications and reduce financial risks before processing credit card approvals.

7. Continuous Model Retraining

The machine learning model can be periodically retrained using newly collected banking data to maintain prediction accuracy and adapt to changing customer behavior.

8. API-Based Prediction Service

The trained machine learning model can be converted into a REST API, allowing other banking applications and websites to use the prediction service efficiently.

15.2 Appendix

Project Source Code

The project was developed using the following technologies:

- Python
- Flask
- HTML
- CSS
- JavaScript
- Scikit-learn
- XGBoost
- Pandas
- NumPy
- Matplotlib
- Seaborn

Dataset

Source: Kaggle
Dataset Files

- application_record.csv
- credit_record.csv

These datasets contain applicant demographic information and historical credit repayment records used to train the machine learning model.

GitHub Repository

GitHub Repository: https://github.com/yellamandakiran/Artificial-Intelligence-and-Machine-Learning

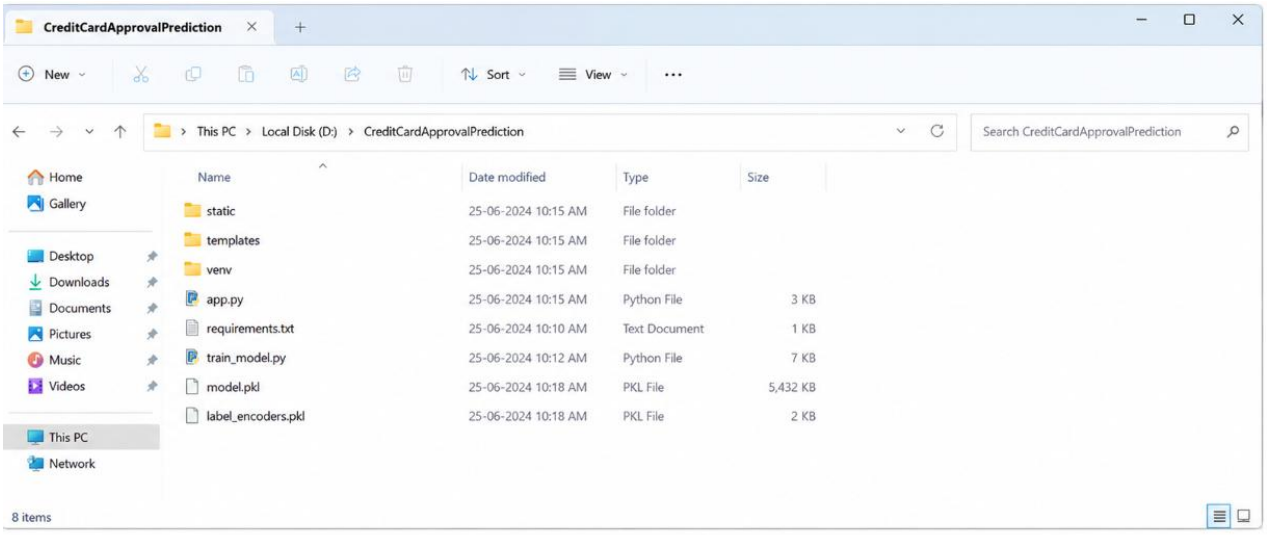
Deployment Process

Step 1: Download / Prepare the Project Folder

Procedure

1. Download the complete project from the GitHub repository or copy the project folder to your local computer.
2. Store the project inside a dedicated folder named:
   CreditCardApprovalPrediction

3. Verify that the project contains all required files and folders, including:
   - dataset/
   - static/
   - templates/
   - app.py
   - train_model.py
   - model.pkl
   - label_encoders.pkl
   - requirements.txt
   - README.md
4. Ensure that no project files are missing before proceeding with deployment.



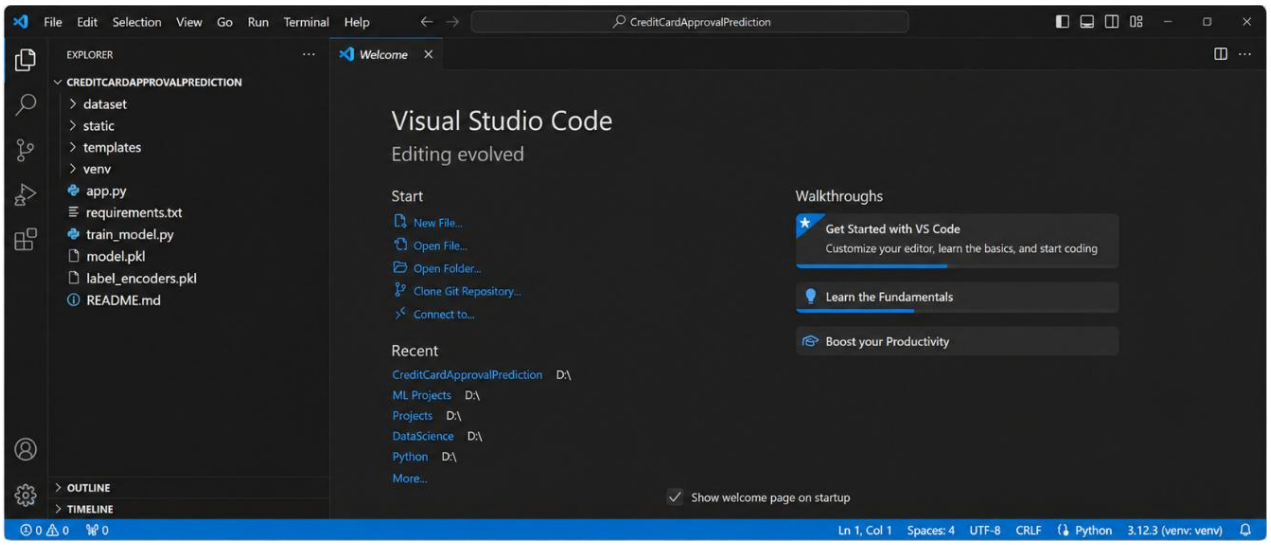
STEP 2: Open the Project in Visual Studio Code

Now, open **Visual Studio Code** and load the project folder.

Click on **File → Open Folder** and select the **"CreditCardApprovalPrediction"** folder.

The project files and folders will be displayed in the **Explorer** panel.

This is where you will write code, run commands, and manage the project.



STEP 3: Create and Activate Virtual Environment

Virtual environment is used to install the required packages for the project.

Open the **Terminal** in **VS Code** and run the following commands:

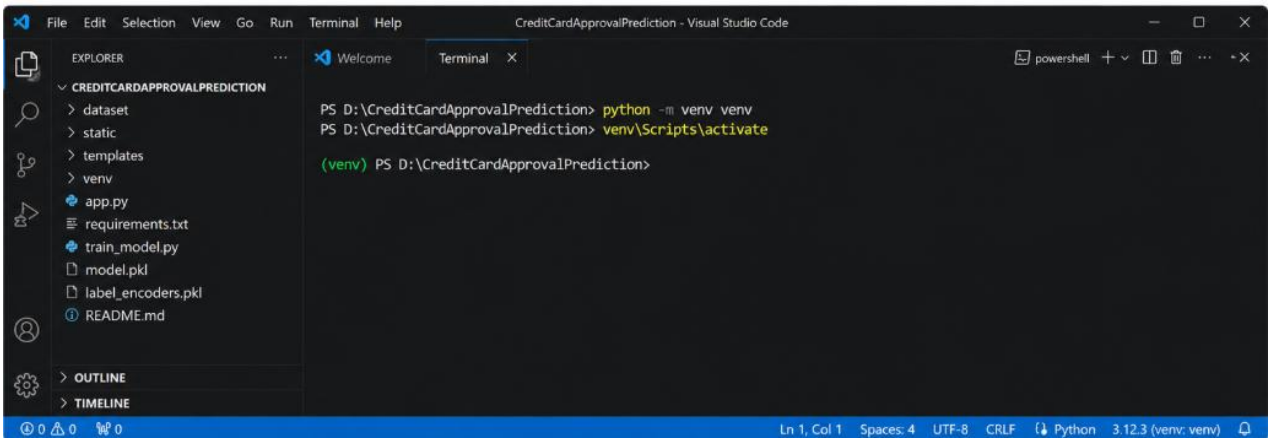
1. Create virtual environment:

python -m venv venv

2. Activate virtual environment (Windows):

venv\Scripts\activate

After activation, you will see **"(venv)"** at the beginning of the terminal, which indicates that the virtual environment is active.



STEP 4: Install Required Packages

Now install all the required Python packages listed in the **requirements.txt** file.

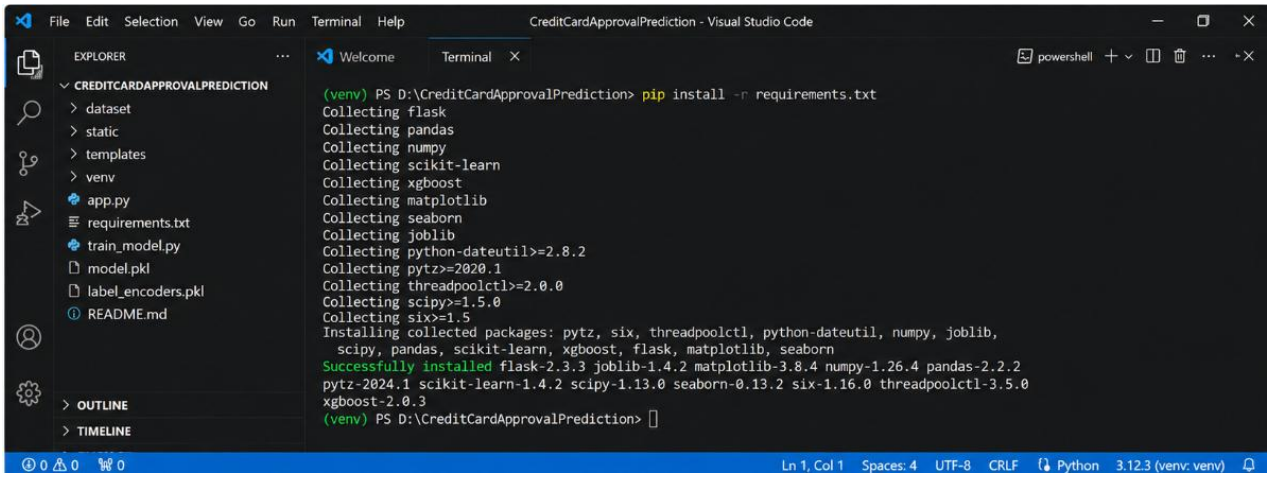
In the **VS Code** terminal, run the following command:

pip install -r requirements.txt

This command will download and install all the dependencies required to run the application.

Wait until the installation is completed successfully.

All the necessary libraries such as **Flask, Pandas, NumPy, Scikit-learn, XGBoost, Matplotlib, Seaborn, and Joblib** will be installed.



STEP 5: Train the Machine Learning Model

Now train the machine learning model using the training script.

In the **VS Code** terminal, run the following command:

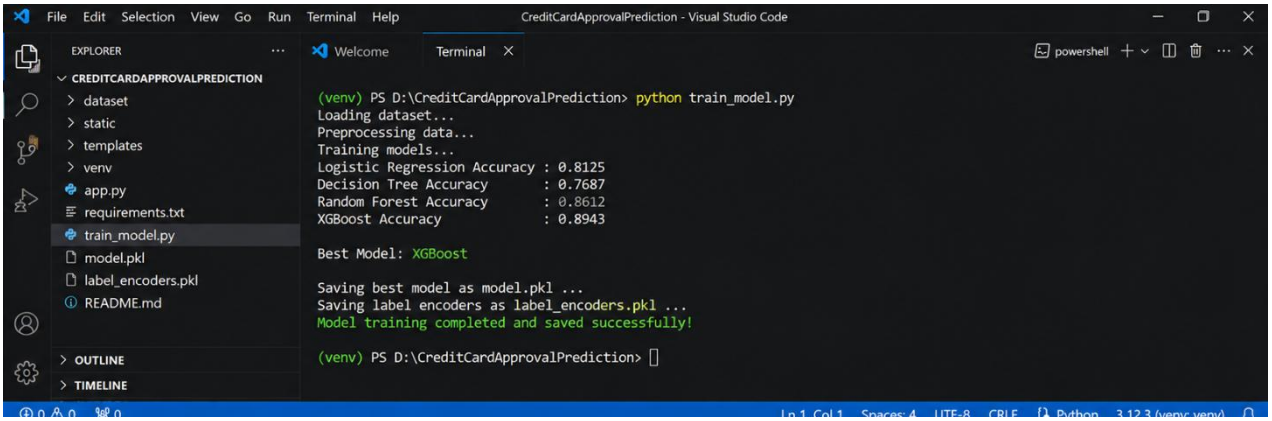
python train_model.py

This script will load the dataset, preprocess the data, train multiple machine learning models, evaluate their performance, and save the best model as **"model.pkl"**.

It will also save the label encoders as **"label_encoders.pkl"**.

Wait until you see the message **"Model training completed and saved successfully!"**

This means the model is trained and ready to make predictions.



STEP 6: Run the Flask Web Application

Now start the **Flask web application**.

In the **VS Code** terminal, run the following command:

python app.py

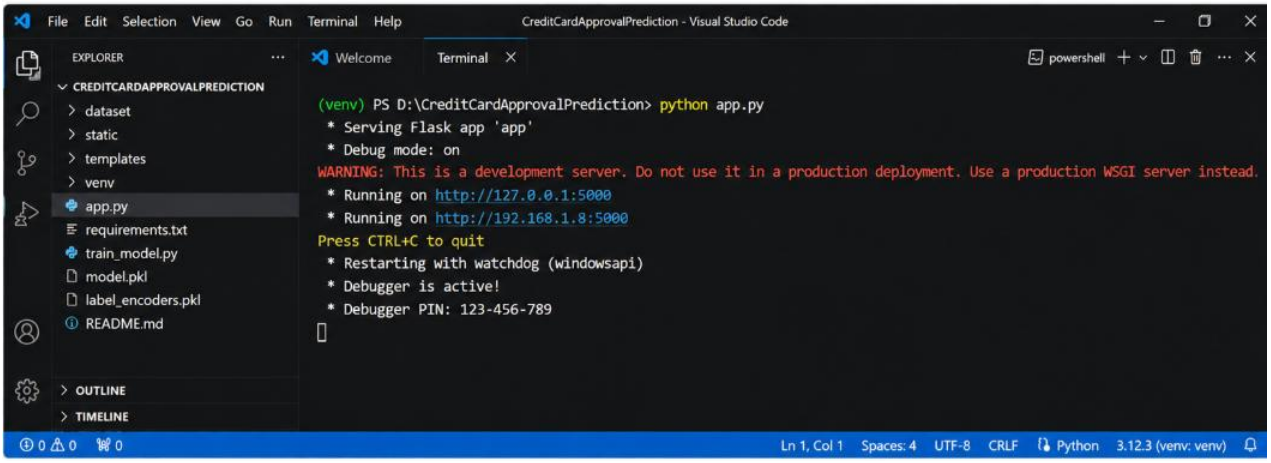
If everything is set up correctly, you will see a message like:

* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

Open your web browser and go to:

http://127.0.0.1:5000/

You will see the **Credit Card Approval Prediction** web interface.



STEP 7: Test the Web Application

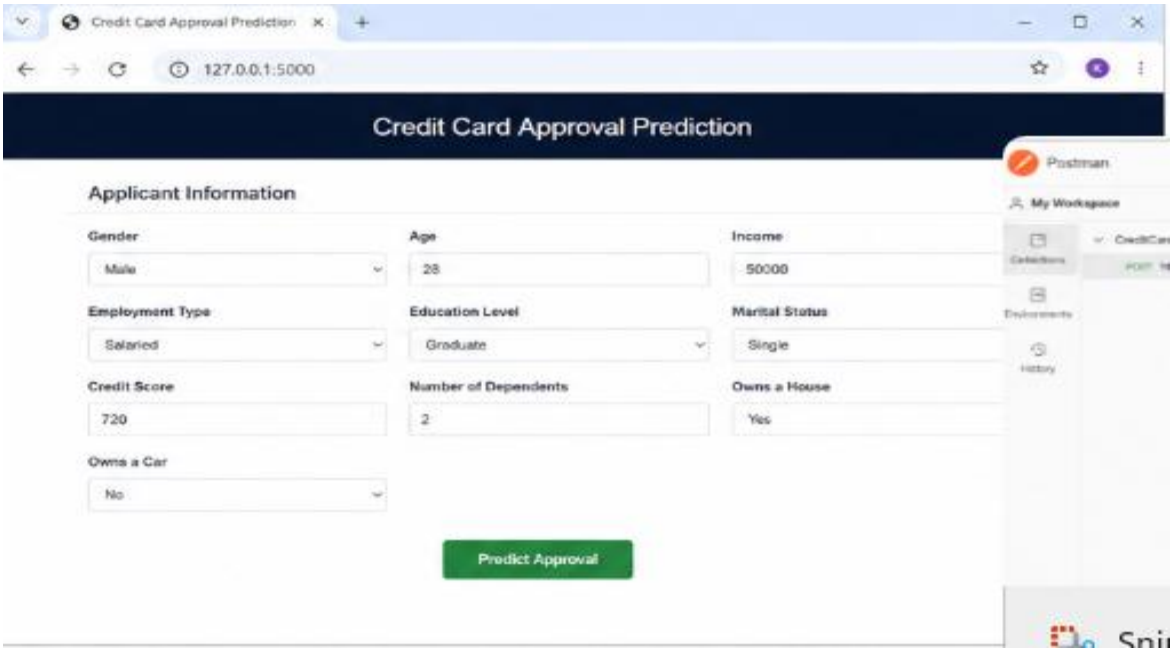
Open your web browser and go to the following URL:

http://127.0.0.1:5000/

You will see the **Credit Card Approval Prediction** web interface.

Enter the applicant details in the form and click on the **"Predict"** button.

The application will display the prediction result as **"Approved"** or **"Rejected"**.



STEP 8: View the Prediction Result

After entering all the applicant details, click the **"Predict"** button.

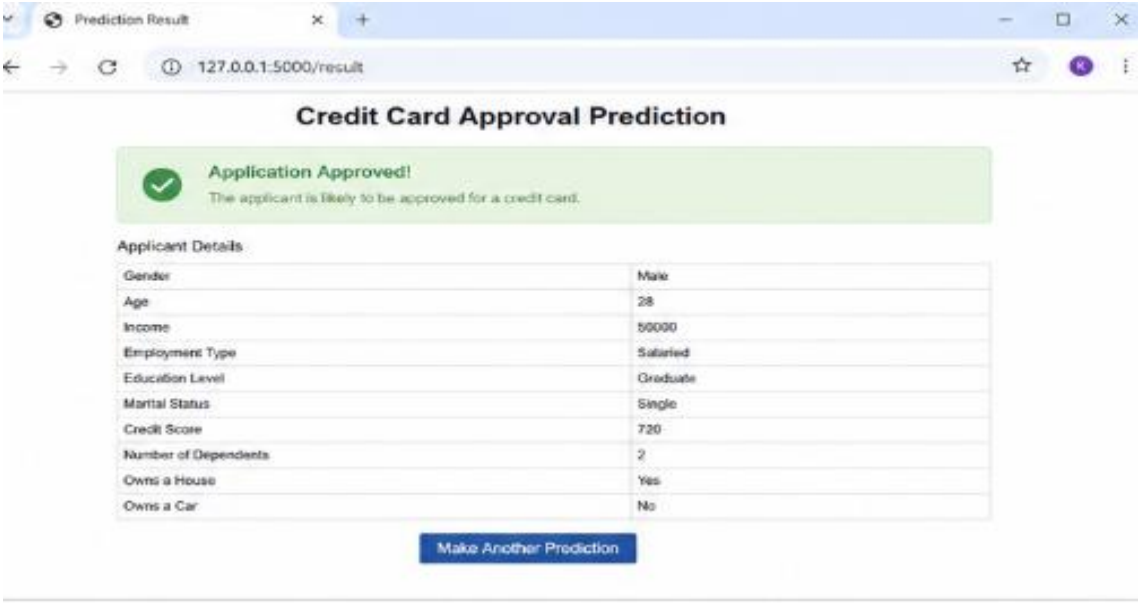
The application processes the input data and sends it to the trained machine learning model for prediction.

Within a few seconds, the system displays the prediction result on the screen.

If the applicant satisfies the required eligibility criteria, the result will be displayed as **"APPROVED"**.

Otherwise, the result will be displayed as **"REJECTED"**.

Verify that the prediction result is displayed correctly before proceeding to the next step.



STEP 9: Close the Application

After verifying the prediction result, return to the **Visual Studio Code** terminal where the Flask application is running.

Press the following keyboard shortcut to stop the Flask server:

CTRL + C

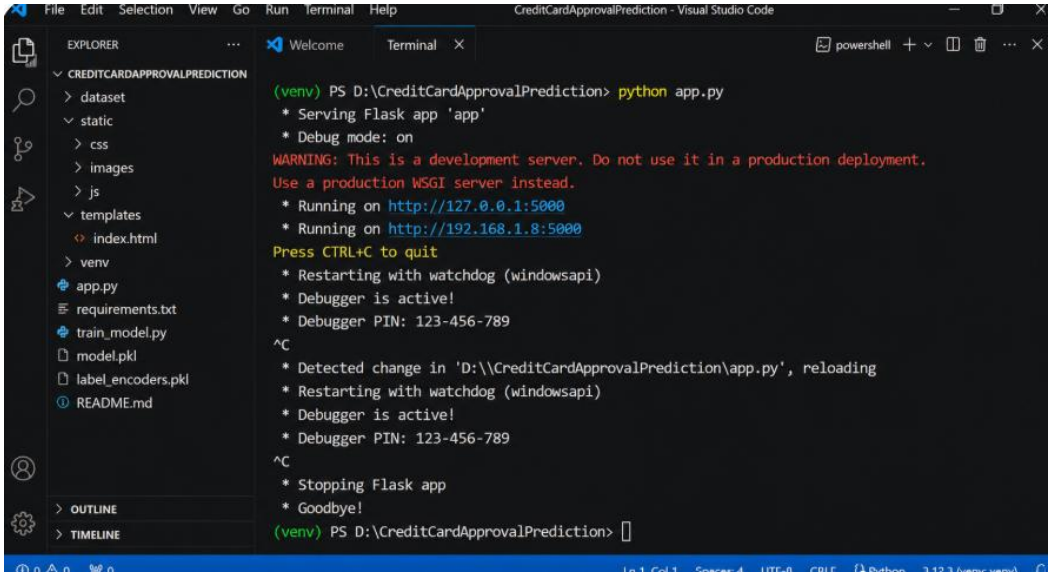
The terminal will stop the Flask application and return to the command prompt.

After closing the application, ensure that all project files, trained models, and datasets are saved successfully.

You can restart the application at any time by executing the following command:

python app.py

The Credit Card Approval Prediction System has now been successfully developed, tested, and executed on the local system.



15.3 References

1. Kaggle – Credit Card Approval Prediction Dataset
   https://www.kaggle.com/
2. Scikit-learn Documentation
   https://scikit-learn.org/
3. XGBoost Documentation
   https://xgboost.readthedocs.io/
4. Flask Documentation
   https://flask.palletsprojects.com/
5. Pandas Documentation
   https://pandas.pydata.org/
6. IBM Watson Machine Learning Documentation
   https://www.ibm.com/cloud/watson-machine-learning
7. Matplotlib Documentation
   https://matplotlib.org/
8. Seaborn Documentation
   https://seaborn.pydata.org/